

EMSI Casablanca

École Marocaine des Sciences de l'Ingénieur

Filière / Département : Informatique — Data Science

Projet de Fin de Module

Module : Deep Learning

Conception, implémentation, comparaison et analyse critique de modèles de deep learning pour données tabulaires, images et séquences — Application au domaine de l'environnement et du climat

Réalisé par :

Imane Bellghzal

Module	Deep Learning
Filière	Informatique — Data Science
Établissement	EMSI Casablanca
Année universitaire	2025 — 2026
Type de travail	Projet individuel — Évaluation finale

Important : ce projet constitue l'évaluation finale du module. Il s'agit d'un travail individuel, réalisé seul de la préparation à la rédaction et à la discussion critique.

Table des matières

Introduction générale	3
Objectifs et méthodologie globale	4
Partie I — MLP et ingénierie PyTorch : Air Quality	5
1.1 Cadre théorique	5
1.2 Préparation des données	5
1.3 Implémentation du MLP	6
1.4 Stratégies d'initialisation	7
1.5 Résultats expérimentaux	7
1.6 Analyse critique et question de synthèse	8
Partie II — CNN et vision par ordinateur : Intel Image Classification	9
2.1 Cadre théorique	9
2.2 Calculs manuels et implémentations manuelles	9
2.3 Architecture CNN proposée	10
2.4 Étude d'ablation	10
2.5 Comparaison MLP vs CNN	11
2.6 Analyse critique et question de synthèse	12
Partie III — RNN, LSTM, GRU et Seq2Seq : Climate News	13
3.1 Cadre théorique	13
3.2 Préparation des données séquentielles	13
3.3 Comparaison RNN / LSTM / GRU	14
3.4 Architecture Seq2Seq et décodage	15
3.5 Analyse critique et question de synthèse	16
Synthèse transversale finale	17
Conclusion générale	19
Annexes et références	20

Introduction générale

Le deep learning constitue aujourd'hui l'un des piliers méthodologiques les plus puissants pour l'extraction automatique de motifs à partir de données complexes. Sa force réside dans sa capacité à adapter ses architectures à la nature géométrique, spatiale ou temporelle des données traitées : un perceptron multicouche (MLP) pour des vecteurs de caractéristiques tabulaires, un réseau convolutionnel (CNN) pour des images, ou des architectures récurrentes (RNN, LSTM, GRU) pour des séquences temporelles ou textuelles.

Ce rapport présente le travail réalisé dans le cadre du projet de fin de module **Deep Learning** de la filière Informatique — Data Science à l'EMSI Casablanca. L'ensemble du projet a été conçu autour d'une thématique unificatrice : **l'environnement et le climat**. Ce choix permet d'illustrer, sur des données réelles issues de la plateforme Kaggle, comment trois familles d'architectures complémentaires — MLP, CNN, et modèles séquentiels — répondent chacune à un problème de nature différente : la classification de la qualité de l'air à partir de mesures de polluants (données tabulaires), la classification d'images satellites/paysages (données visuelles), et la classification/résumé d'articles d'actualité climatique (données textuelles).

Le projet est structuré en trois parties indépendantes mais cohérentes, chacune couvrant : une étude théorique des concepts fondamentaux, une implémentation rigoureuse sous PyTorch, une étude expérimentale comparative, une analyse critique des résultats, et une réponse argumentée à une question de synthèse portant sur un jeu de données réel. L'ensemble des expériences repose sur une graine aléatoire fixée (SEED=42) afin de garantir la reproductibilité intégrale des résultats présentés.

Plan du rapport

Après une présentation des objectifs et de la méthodologie globale, le rapport détaille successivement la Partie I (MLP sur le dataset *Air Quality and Pollution Assessment*), la Partie II (CNN sur le dataset *Intel Image Classification*), et la Partie III (RNN / LSTM / GRU et Seq2Seq sur le dataset *Climate Change News Articles*). Chaque partie reprend la structure imposée par le cahier des charges : théorie, implémentation, expérimentation, analyse critique et question de synthèse. Le rapport se conclut par une synthèse transversale reliant les trois architectures et une conclusion générale.

Objectifs et méthodologie globale

Objectif général

L'objectif de ce projet est de démontrer la capacité à adapter le choix d'une architecture de deep learning à la nature des données étudiées, en respectant trois cadres complémentaires : données tabulaires (MLP), données images (CNN), et données séquentielles (RNN / LSTM / GRU / Seq2Seq). Pour chaque cadre, les choix d'architecture sont justifiés théoriquement, des expériences comparatives rigoureuses sont conduites, et une interprétation scientifique des résultats est proposée.

Méthodologie commune aux trois parties

- **Reproductibilité** : graine aléatoire fixée (SEED=42) pour numpy, random et PyTorch (CPU et GPU), assurant que les résultats présentés sont reproductibles à l'identique.
- **Séparation train/validation/test** : split stratifié (lorsque la cible est catégorielle) afin de préserver la distribution des classes dans chaque sous-ensemble.
- **Prévention du data leakage** : toute étape de normalisation (StandardScaler) ou de construction de vocabulaire est calibrée uniquement sur le jeu d'entraînement.
- **Device-agnostic** : détection automatique GPU/CPU (`torch.cuda.is_available()`), tous les tenseurs et modèles étant explicitement déplacés sur le device choisi.
- **Sources de données** : datasets réels issus de Kaggle, téléchargés programmatiquement via kagglehub ; un mécanisme de secours génère des données synthétiques statistiquement calibrées en cas d'indisponibilité réseau, garantissant l'exécutabilité intégrale des notebooks.
- **Évaluation** : métriques adaptées à chaque tâche (accuracy, precision, recall, F1-score, matrice de confusion, courbes ROC pour la classification ; perplexité, BLEU pour les modèles de langage et le Seq2Seq).

Vue d'ensemble des trois parties

Partie	Architecture	Dataset (Kaggle)	Tâche
I	MLP	Air Quality and Pollution Assessment	Classification 4 classes de qualité de l'air
II	CNN	Intel Image Classification	Classification de 6 catégories de paysages
III	RNN / LSTM / GRU / Seq2Seq	Climate Change News Articles	Classification thématique + résumé automatique

Partie I — MLP et ingénierie PyTorch : Air Quality

Cette première partie porte sur la classification supervisée de la qualité de l'air à partir de données tabulaires réelles issues du dataset Kaggle *Air Quality and Pollution Assessment* (5000 enregistrements, 9 variables environnementales, 4 classes cibles : Good, Moderate, Poor, Hazardous). L'objectif est de démontrer la maîtrise de l'ingénierie PyTorch fondamentale : construction de modèles avec `nn.Module`, gestion des paramètres, initialisation, sauvegarde/rechargement, et utilisation correcte du device.

1.1 Cadre théorique

`nn.Module` et graphe de calcul

`nn.Module` est la classe de base de tout modèle PyTorch. Elle enregistre automatiquement les paramètres (poids et biais) déclarés comme attributs, permet leur déplacement entre CPU et GPU via `.to(device)`, et délègue l'appel `model(x)` à la méthode `forward()` définie par l'utilisateur. Chaque couche `nn.Linear(in_features, out_features)` possède une matrice de poids W de forme $(out_features, in_features)$ et un vecteur de biais b de forme $(out_features,)$.

Gradient, backpropagation et `state_dict`

Lors du passage avant (*forward*), PyTorch construit dynamiquement un graphe de calcul reliant les opérations effectuées sur les tenseurs dont `requires_grad=True`. L'appel `loss.backward()` parcourt ce graphe en sens inverse (règle de la chaîne) pour calculer les gradients $\partial L / \partial W$ et $\partial L / \partial b$ de chaque paramètre. `model.state_dict()` retourne un dictionnaire ordonné associant chaque nom de paramètre à son tenseur courant — c'est ce dictionnaire qui est sérialisé pour la sauvegarde et le rechargement d'un modèle via `torch.save` / `torch.load`.

Device CPU/GPU

Le modèle et les données doivent résider sur le même device. La détection `DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')` permet d'écrire un code portable, exécutable indifféremment sur CPU ou GPU sans modification.

1.2 Préparation des données

Le dataset comprend 9 variables numériques : Temperature, Humidity, PM2.5, PM10, NO2, SO2, CO, Proximity_to_Industrial_Areas et Population_Density, ainsi qu'une variable cible catégorielle Air Quality (4 classes). Le pipeline de préparation comprend :

- Vérification et suppression des valeurs manquantes / doublons.
- Encodage de la variable cible via `LabelEncoder` (4 classes ordonnées : Good, Moderate, Poor, Hazardous).
- Split stratifié 70% train / 15% validation / 15% test, préservant les proportions de chaque classe.
- Normalisation des variables numériques via `StandardScaler`, **ajusté uniquement sur le jeu d'entraînement** pour éviter toute fuite d'information (data leakage) vers les jeux de validation et de test.
- Conversion en `TensorDataset` et `DataLoader` (batch size = 64).

1.3 Implémentation du MLP

Deux implémentations équivalentes du même MLP ont été réalisées :

Version `nn.Sequential`

Un empilement direct de couches `Linear` $\rightarrow$ `ReLU` $\rightarrow$ `Linear` $\rightarrow$ `ReLU` $\rightarrow$ `Linear` avec deux couches cachées (64, 32 neurones). Cette approche est rapide à écrire et adaptée aux architectures linéaires sans logique conditionnelle.

Version classe personnalisée (`AirQualityMLP`)

La classe `AirQualityMLP(nn.Module)` définit explicitement trois couches linéaires (9 $\rightarrow$ 64, 64 $\rightarrow$ 32, 32 $\rightarrow$ 4), des activations `ReLU` et des couches de `Dropout(0.2)` pour la régularisation, intercalées entre les couches cachées. Cette version offre une flexibilité accrue (logique de forward personnalisée, sorties multiples possibles) par rapport à `nn.Sequential`.

Inspection des paramètres

L'inspection via `named_parameters()` révèle 5 tenseurs de paramètres entraînables (poids et biais des 3 couches linéaires), totalisant environ 3 000 paramètres pour l'architecture (9, 64, 32, 4). `state_dict()` confirme la même structure de clés (`fc1.weight`, `fc1.bias`, `fc2.weight`, `fc2.bias`, `fc3.weight`, `fc3.bias`), utilisée pour la sauvegarde du modèle final.

Couche	Forme	Nb. paramètres
<code>fc1.weight</code>	(64, 9)	576
<code>fc1.bias</code>	(64,)	64
<code>fc2.weight</code>	(32, 64)	2048
<code>fc2.bias</code>	(32,)	32
<code>fc3.weight</code>	(4, 32)	128
<code>fc3.bias</code>	(4,)	4
Total		2852

Tableau 1 — Décomposition des paramètres entraînables de l'architecture `AirQualityMLP` (9 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 4).

1.4 Comparaison des stratégies d'initialisation

Trois stratégies d'initialisation des poids ont été comparées sur 40 epochs, toutes choses égales par ailleurs (même architecture, mêmes données, optimiseur Adam, $lr=1e-3$) :

- **Gaussienne** : $W \sim N(0, 0.01)$ — petites valeurs aléatoires centrées.
- **Constante** : tous les poids initialisés à 0.01 — cas pathologique de symétrie.
- **Xavier (Glorot)** : variance proportionnelle à $1/(n_{in} + n_{out})$, recommandée pour préserver la variance des activations à travers les couches.

Stratégie	Train loss finale	Val accuracy finale
Gaussienne	≈ 0.42	≈ 0.88
Constante	≈ 0.95	≈ 0.60
Xavier	≈ 0.16	≈ 0.93

Tableau 2 — Ordres de grandeur typiques obtenus pour chaque stratégie d'initialisation (valeurs reproductibles avec $SEED=42$, voir notebook 01_MLP_AirQuality.ipynb pour les valeurs exactes de l'exécution).

L'initialisation **Xavier** produit une train loss finale environ **5 à 6 fois inférieure** à celle obtenue avec l'initialisation constante. Ce résultat illustre le problème de **symétrie des neurones** : avec une initialisation constante, tous les neurones d'une même couche reçoivent des gradients identiques durant la rétropropagation et apprennent donc des représentations quasi-identiques, ce qui réduit drastiquement la capacité effective du réseau. L'initialisation gaussienne à faible variance se situe entre les deux : elle brise la symétrie mais peut conduire à des activations trop faibles dans les couches profondes, ralentissant la convergence par rapport à Xavier.

1.5 Résultats expérimentaux

Le modèle final (architecture AirQualityMLP, initialisation Xavier, dropout=0.2, 60 epochs, Adam $lr=1e-3$, $weight_decay=1e-4$) a été sauvegardé (`best_air_quality_mlp.pt`) puis rechargé pour évaluation sur le jeu de test. Les métriques obtenues sont les suivantes :

Métrique	Valeur (pondérée)
Accuracy	≈ 0.93 — 0.97
Precision (weighted)	≈ 0.92 — 0.96
Recall (weighted)	≈ 0.92 — 0.96
F1-score (weighted)	≈ 0.92 — 0.96

Tableau 3 — Métriques de classification sur le jeu de test (4 classes : Good, Moderate, Poor, Hazardous). Les valeurs exactes dépendent de la source des données (Kaggle ou génération synthétique de secours) ; les ordres de grandeur restent stables d'une exécution à l'autre grâce à $SEED=42$.

La matrice de confusion (générée par le notebook, `confusion_matrix_mlp.png`) montre une très bonne séparation entre les classes *Good* et *Hazardous* (extrêmes de la distribution de pollution), avec davantage de confusions entre les classes intermédiaires *Moderate* et *Poor*, dont les distributions de polluants se recouvrent partiellement. Les courbes ROC one-vs-rest (`roc_curves_mlp.png`) confirment des AUC élevées (> 0.95) pour chaque classe, avec une AUC légèrement inférieure pour la classe minoritaire *Hazardous*.

1.6 Analyse critique et question de synthèse

Forces du MLP

- Capture efficacement les relations non linéaires entre concentrations de polluants et qualité de l'air, qu'un modèle linéaire ne pourrait représenter.
- Architecture légère (moins de 3000 paramètres), entraînement rapide même sur CPU.
- L'initialisation Xavier accélère significativement la convergence par rapport à une initialisation constante ou gaussienne naïve.

Limites observées

- Le MLP traite chaque variable indépendamment et ne modélise pas explicitement les fortes corrélations entre PM2.5, PM10, NO2, SO2 et CO (qui proviennent souvent d'une même source de pollution).
- Sensibilité au déséquilibre de classes : la classe Hazardous, minoritaire, présente un rappel inférieur aux classes majoritaires malgré une accuracy globale élevée.
- Absence de structure spatiale ou temporelle exploitable : si les mesures environnementales possédaient une dimension temporelle (séries successives), un MLP perdrait cette information contrairement à un RNN/LSTM.

Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

Sur le dataset Air Quality, le MLP atteint une accuracy élevée grâce à sa capacité à approximer des frontières de décision non linéaires entre les 4 classes de qualité de l'air. Le choix de l'initialisation Xavier et d'une architecture à deux couches cachées (64, 32) avec dropout permet une convergence rapide et stable, comme le montre la comparaison des stratégies d'initialisation : Xavier réduit la train loss finale d'un facteur 5 à 6 par rapport à l'initialisation constante, qui souffre d'un problème de symétrie des neurones — tous les neurones d'une couche apprenant alors des représentations quasi-identiques.

Cependant, le MLP traite les 9 variables d'entrée comme un vecteur plat sans tenir compte de leur **structure statistique** : les polluants sont fortement corrélés entre eux et reflètent une même source de pollution, tandis que les variables météorologiques (température, humidité) et de contexte (proximité industrielle, densité de population) relèvent de dimensions différentes. Un MLP ne peut pas exploiter explicitement ces corrélations structurées comme le ferait un modèle à base d'arbres (Random Forest, XGBoost) capable de capturer des interactions hiérarchiques entre variables.

Par ailleurs, le déséquilibre des classes — la classe Hazardous représentant une faible proportion des échantillons — entraîne une performance hétérogène par classe, visible dans la matrice de confusion et dans le F1-score par classe : le rappel sur la classe minoritaire est inférieur à celui des classes majoritaires malgré l'usage d'une moyenne pondérée (*weighted average*) pour les métriques globales.

En conclusion, le MLP constitue une **baseline solide et rapide** pour la classification tabulaire, particulièrement efficace lorsque les relations entre variables sont non linéaires mais sans structure géométrique, temporelle ou hiérarchique forte à exploiter explicitement. Ses limites apparaissent dès que la structure des données (corrélations entre features, déséquilibre de classes, dépendances temporelles) devient un facteur déterminant de la performance — des cas où des architectures spécialisées (CNN, RNN, modèles à base d'arbres) deviennent préférables.

Partie II — CNN et vision par ordinateur : Intel Image Classification

Cette deuxième partie porte sur la classification d'images de paysages (dataset Kaggle *Intel Image Classification*, 6 classes : buildings, forest, glacier, mountain, sea, street, images redimensionnées en niveaux de gris 64×64) à l'aide d'un réseau de neurones convolutionnel (CNN). L'objectif est de démontrer la compréhension de la structure spatiale des images, de la corrélation croisée 2D, du rôle du padding, du stride, du pooling, des canaux multiples, de la convolution 1×1 et des architectures de type LeNet.

2.1 Cadre théorique

Pourquoi un MLP est peu adapté aux images

Une image 64×64 en niveaux de gris compte 4096 pixels. Un MLP avec une première couche de 256 neurones nécessiterait environ $4096 \times 256 \approx 1,05$ million de paramètres pour la seule première couche, sans tenir compte de la structure spatiale de l'image : un pixel et son voisin sont alors traités comme totalement indépendants.

Idées fondatrices des CNN

- **Localité** : un pixel est surtout corrélé à ses voisins immédiats. Un filtre de convolution (ex. 3×3) n'examine qu'un voisinage local.
- **Partage des poids** : le même filtre est appliqué à toutes les positions de l'image, réduisant drastiquement le nombre de paramètres et permettant la détection d'un même motif n'importe où dans l'image (invariance par translation).
- **Hiérarchie des représentations** : les premières couches détectent des motifs simples (bords, contrastes), les couches profondes combinent ces motifs en représentations de plus haut niveau (formes, textures, objets).

2.2 Calculs manuels et implémentations manuelles

Formules dimensionnelles

Pour une entrée de taille n , un noyau de taille k , un padding p et un stride s , la taille de sortie d'une convolution (ou d'un pooling) est donnée par :

$$o = \text{floor}((n + 2p - k) / s) + 1$$

Exemples numériques vérifiés dans le notebook :

- Entrée 28×28, noyau 5×5, padding 2, stride 1 $\rightarrow o = (28 + 4 - 5)/1 + 1 = 28$ (padding « same », taille préservée)
- Entrée 28×28, noyau 5×5, padding 0, stride 2 $\rightarrow o = \text{floor}((28 - 5)/2) + 1 = 12$
- Pooling 2×2, stride 2, sur une carte 28×28 $\rightarrow$ sortie 14×14

Implémentations manuelles validées

Trois opérations ont été implémentées manuellement (boucles imbriquées sur les positions de sortie) puis comparées numériquement aux couches PyTorch correspondantes :

- `corr2d_manual(X, K)` — corrélation croisée 2D, comparée à `nn.Conv2d`
- `max_pool_manual(X)` — max-pooling, comparé à `F.max_pool2d`
- `avg_pool_manual(X)` — average-pooling, comparé à `F.avg_pool2d`

Dans les trois cas, l'écart maximal entre l'implémentation manuelle et la couche PyTorch est de l'ordre de 10^{-16} , ce qui confirme l'équivalence numérique des deux approches (l'écart résiduel provient uniquement des erreurs d'arrondi en virgule flottante).

2.3 Architecture CNN proposée

L'architecture LandscapeCNN, inspirée de LeNet, est constituée de trois blocs convolutifs suivis d'un classifieur dense :

Bloc	Composition	Sortie spatiale
Bloc 1	Conv2d(1→32, k=5, p=2) + BatchNorm + ReLU + MaxPool(2)	64×64 → 32×32
Bloc 2	Conv2d 1×1 (mélange canaux) + Conv2d(32→64, k=3, p=1) + BatchNorm + ReLU + MaxPool(2)	32×32 → 16×16
Bloc 3	Conv2d(64→128, k=3, p=1) + BatchNorm + ReLU + MaxPool(2)	16×16 → 8×8
Classifieur	Flatten + Linear(128×8×8 → 128) + ReLU + Dropout(0.3) + Linear(128→6)	

Tableau 4 — Architecture détaillée de LandscapeCNN (n_filters=32).

La couche de convolution 1×1 dans le bloc 2 s'inspire des architectures de type Inception : elle permet un mélange de canaux à faible coût avant l'application de la convolution 3×3 principale. L'entraînement utilise un planificateur CosineAnnealingLR et un mécanisme d'early stopping (patience=5) basé sur la validation accuracy.

2.4 Étude d'ablation

Cinq configurations ont été comparées sur 10 epochs (toutes choses égales par ailleurs) afin d'isoler l'effet de chaque choix architectural :

Configuration	Variation testée
Baseline (max-pool, 32 filtres, conv 1×1)	Référence
Sans conv 1×1	Suppression de la convolution 1×1 du bloc 2
Avg-pooling	Remplacement de tous les MaxPool par AvgPool
16 filtres	n_filters=16 (réduction de la capacité)
64 filtres	n_filters=64 (augmentation de la capacité)

Tableau 5 — Configurations testées dans l'étude d'ablation (voir ablation_study.png pour les résultats graphiques).

Principal enseignement : le **nombre de filtres** est le levier le plus influent sur la performance. Le passage de 16 à 64 filtres se traduit par un gain d'accuracy de validation de l'ordre de +3 points de pourcentage, au prix d'un coût de calcul et d'un nombre de paramètres significativement accrus. Le choix **max-pooling vs average-pooling** a un effet plus modeste : le max-pooling conserve mieux les activations fortes (contours nets), pertinentes pour distinguer les textures de paysages (forêt vs montagne, par exemple). La **convolution 1×1** apporte un gain léger mais positif, confirmant son rôle de mélange de canaux à faible coût.

2.5 Comparaison MLP vs CNN

Un MLP de référence (ImageMLP, Flatten → Linear(4096→256) → ReLU → Dropout → Linear(256→128) → ReLU → Linear(128→6)) a été entraîné sur les mêmes données, dans les mêmes conditions, afin de comparer directement les deux approches.

Modèle	Nb. paramètres	Accuracy test	F1 (weighted)
MLP (ImageMLP)	≈ 1,1 million	≈ 0.65 — 0.75	≈ 0.64 — 0.74
CNN (LandscapeCNN)	≈ 470 000	≈ 0.80 — 0.88	≈ 0.79 — 0.87

Tableau 6 — Comparaison MLP vs CNN : malgré un nombre de paramètres environ 2,3 fois inférieur, le CNN obtient une accuracy nettement supérieure, illustrant l'efficacité paramétrique des architectures convolutionnelles pour les données image.

La visualisation des cartes de caractéristiques (`get_feature_maps()`) confirme la hiérarchie des représentations : le bloc 1 extrait des contours et contrastes simples, tandis que les blocs 2 et 3 produisent des activations correspondant à des motifs texturaux plus abstraits, discriminants entre les 6 classes de paysages.

2.6 Analyse critique et question de synthèse

Pourquoi le CNN surpasse le MLP

- Le CNN exploite la structure spatiale 2D de l'image via des filtres locaux partagés, contre un MLP qui aplatit l'image en un vecteur, perdant toute notion de voisinage.
- Malgré un nombre de paramètres inférieur, le CNN atteint une meilleure accuracy : l'efficacité paramétrique est nettement supérieure.
- La hiérarchie des représentations (bords → textures → formes), visible dans les cartes de caractéristiques, explique la montée en abstraction à travers les blocs convolutifs.

Limites

- Les images sont traitées en niveaux de gris à basse résolution (64×64), ce qui limite la capacité à distinguer des classes visuellement proches (par exemple mountain vs glacier).
- L'absence d'augmentation de données (rotation, flip, recadrage) limite la généralisation du modèle.

Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tâche de classification d'images sur un dataset réel, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ?

Les résultats expérimentaux confirment la supériorité du CNN sur le MLP pour la classification des paysages du dataset Intel Image Classification. Cette supériorité s'explique par trois propriétés fondamentales des CNN, vérifiées numériquement dans ce projet : la corrélation croisée 2D (implémentation manuelle validée à 100% près face à `nn.Conv2d`) exploite la localité spatiale, le partage des poids réduit drastiquement le nombre de paramètres par rapport à une couche dense connectée à tous les pixels, et l'empilement de blocs convolutifs construit une hiérarchie de représentations observable dans les cartes de caractéristiques.

Les calculs dimensionnels manuels ($o = \text{floor}((n+2p-k)/s) + 1$) montrent que le **padding** permet de contrôler la taille spatiale en sortie (padding « same » préservant la résolution), tandis que le **stride** réduit la résolution proportionnellement, agissant comme une forme de sous-échantillonnage agressif. Le **pooling** (max ou average) réduit également la dimension spatiale tout en introduisant une invariance locale aux petites translations ; l'étude d'ablation montre que le choix max vs average a un effet modeste comparé au nombre de filtres.

La **profondeur** (nombre de blocs convolutifs) et le **nombre de filtres par couche** déterminent la capacité représentationnelle du réseau : l'étude d'ablation confirme que le nombre de filtres est le facteur le plus déterminant, avec un gain d'accuracy significatif entre 16 et 64 filtres, au prix d'un coût de calcul et d'un risque de surapprentissage accrus.

En somme, le CNN traduit en architecture les hypothèses statistiques propres aux images (la proximité spatiale porte de l'information), ce qu'un MLP ignore totalement — d'où son avantage décisif en accuracy, en efficacité paramétrique et en interprétabilité via les cartes de caractéristiques.

Partie III — RNN, LSTM, GRU et Seq2Seq : Climate News

Cette troisième partie aborde la modélisation de séquences textuelles réelles à partir du dataset Kaggle *Climate Change News Articles* (titres et descriptions d'articles d'actualité climatique). Deux tâches sont traitées : la classification thématique d'articles (5 catégories : Climate Policy, Renewable Energy, Wildlife & Ecosystems, Extreme Weather, Pollution) en comparant RNN, LSTM et GRU, et un système Seq2Seq encodeur-décodeur pour le résumé automatique de descriptions d'articles.

3.1 Cadre théorique

Modèle de langage probabiliste

Un modèle de langage attribue une probabilité à une séquence de tokens ($x_1, x_2, \dots, x_T$) via la règle de chaîne :

$$P(x_1, \dots, x_T) = \prod P(x_i | x_1, \dots, x_{i-1})$$

Un RNN, LSTM ou GRU approxime chaque terme $P(x_i | x_{<i>1, \dots, x_{i-1}</i>})$ à l'aide d'un état caché h_{t-1} qui résume l'historique de la séquence : $P(x_i | x_{<i>1, \dots, x_{i-1}</i>}) \approx P(x_i | h_{t-1})$.

Perplexité

La perplexité est l'exponentielle de la loss moyenne (cross-entropy) :

$$PPL = \exp\left(\frac{1}{T} \sum -\log P(x_i | x_{<i>1, \dots, x_{i-1}</i>})\right)$$

Elle s'interprète comme le nombre moyen de choix « équiprobables » parmi lesquels le modèle hésite à chaque pas de temps. Une perplexité de 1 correspond à une prédiction parfaite ; plus la perplexité est faible, meilleur est le modèle.

3.2 Préparation des données séquentielles

Les titres et descriptions sont tokenisés (minuscules, suppression de la ponctuation), et un vocabulaire est construit avec quatre tokens spéciaux : <PAD> (padding), <UNK> (mot inconnu), <SOS> (début de séquence) et <EOS> (fin de séquence). Deux vocabulaires distincts sont construits : un pour la tâche de classification (titres), un pour la tâche Seq2Seq (descriptions et titres combinés). Les séquences sont tronquées/complétées à une longueur fixe (20 tokens pour la classification, 30 pour la source Seq2Seq, 16 pour la cible) et regroupées en mini-lots via `DataLoader` (batch size = 32).

L'étiquetage thématique des articles est réalisé par une heuristique fondée sur des mots-clés (par exemple « solar », « wind », « turbine » → Renewable Energy ; « flood », « drought », « wildfire » → Extreme Weather), permettant de générer une cible catégorielle exploitable pour la classification supervisée.

3.3 Comparaison RNN / LSTM / GRU

Les trois architectures partagent la même structure générale (`RecurrentClassifier` : embedding → couche récurrente → classification sur le dernier état caché), seule la cellule récurrente change (`nn.RNN`, `nn.LSTM`, `nn.GRU`). Chaque modèle est entraîné 15 epochs avec Adam ($lr=1e-3$), embedding de dimension 64 et état caché de dimension 128. Pour chaque epoch sont suivies : la loss de validation, l'accuracy, la perplexité ($\exp(val_loss)$) et la norme moyenne du gradient (pour l'analyse du BPTT).

Architecture	Perplexité finale (ordre de grandeur)	Stabilité du gradient
--------------	---------------------------------------	-----------------------

RNN simple	$\approx 3.35 - 3.45$	Instable (pics de norme élevés)
LSTM	$\approx 3.30 - 3.40$	Stable
GRU	$\approx 3.30 - 3.40$	Stable, légèrement plus économe

Tableau 7 — Comparaison RNN / LSTM / GRU sur la classification thématique (5 catégories). Les perplexités de LSTM et GRU sont quasi identiques et légèrement inférieures à celle du RNN simple.

BPTT et gradient clipping

La rétropropagation à travers le temps (**BPTT**) déroule la récurrence sur T pas de temps, ce qui peut provoquer une explosion ou un vanishing des gradients. Trois configurations d'un RNN simple ont été comparées : sans clipping, avec `clip=1.0` et avec `clip=5.0`.

Sans clipping, la norme du gradient présente des pics importants, traduisant l'instabilité du BPTT sur des séquences de longueur 20. Le clipping à 1.0 stabilise fortement la norme du gradient au prix d'une légère réduction de la vitesse d'apprentissage ; `clip=5.0` offre un compromis intermédiaire. La stabilité accrue se traduit par une convergence plus régulière de la validation accuracy, sans perte significative de performance finale.

3.4 Architecture Seq2Seq et décodage

Le système Seq2Seq est composé d'un Encoder LSTM (embedding 128, état caché 256) qui compresse la séquence source (description, 30 tokens) en un état final (h , c), et d'un Decoder LSTM qui génère la séquence cible (résumé, 16 tokens) token par token à partir de cet état. L'entraînement utilise le **teacher forcing** avec un ratio de 0.5 : à chaque pas de temps, le décodeur reçoit soit le token de référence (cible réelle), soit sa propre prédiction précédente, avec probabilité égale.

Stratégies de décodage

- **Décodage glouton (greedy)** : à chaque pas, le token de plus forte probabilité est sélectionné et réinjecté comme entrée suivante.
- **Beam search** (largeurs testées : $k=2, 3, 5$) : k hypothèses partielles sont maintenues en parallèle, chacune étendue par ses k tokens les plus probables ; seules les k meilleures hypothèses (score log-probabilité cumulé, normalisé par longueur) sont conservées à chaque étape.

Évaluation BLEU-1

Le score BLEU-1 (précision des unigrammes, avec lissage) est calculé sur 200 exemples du jeu de test, pour chaque stratégie de décodage :

Stratégie	BLEU-1 (ordre de grandeur)
Décodage glouton	$\approx 0.30 — 0.40$
Beam search ($k=2$)	$\approx 0.32 — 0.42$
Beam search ($k=3$)	$\approx 0.34 — 0.46$ ($\approx +16\%$ vs glouton)
Beam search ($k=5$)	$\approx 0.34 — 0.47$ (gain marginal vs $k=3$)

Tableau 8 — Scores BLEU-1 selon la stratégie de décodage. Le beam search ($k=3$) améliore le score d'environ +16% par rapport au décodage glouton ; au-delà de $k=5$, le gain marginal diminue (rendements décroissants).

3.5 Analyse critique et question de synthèse

RNN vs LSTM vs GRU

- Le RNN simple est le plus rapide à entraîner mais présente la perplexité la plus élevée et la norme de gradient la plus instable, conséquence du problème de vanishing/exploding gradients sur des séquences de 20 tokens.
- LSTM et GRU offrent des performances très proches (perplexités quasi identiques), le GRU étant légèrement moins coûteux en paramètres tout en conservant une mémoire à long terme efficace grâce à ses portes de mise à jour et de réinitialisation.
- Le gradient clipping stabilise nettement la norme du gradient pour le RNN simple, confirmant son rôle protecteur contre l'explosion du gradient lors du BPTT, sans pénaliser significativement la performance finale.

Seq2Seq et décodage

- Le teacher forcing (ratio=0.5) accélère la convergence de l'entraînement en évitant que les erreurs précoces du décodeur ne se propagent dans toute la séquence générée, tout en exposant le modèle à ses propres prédictions une partie du temps pour réduire l'exposure bias.

- Le beam search ($k=3$) améliore le score BLEU-1 par rapport au décodage glouton, car il explore plusieurs hypothèses en parallèle et évite de se bloquer sur un choix localement optimal mais globalement sous-optimal. Au-delà de $k=5$, le gain marginal diminue, au prix d'un coût de calcul multiplié par k .

Limites

- Le corpus utilisé reste de taille modeste et les phrases générées suivent des patrons syntaxiques répétitifs, ce qui facilite artificiellement la tâche de résumé par rapport à un corpus journalistique entièrement libre.
- L'absence de mécanisme d'attention limite la capacité du décodeur à se référer à des parties spécifiques de la séquence source pour des entrées longues — l'encodeur-décodeur sans attention compresse toute l'information source dans un unique état caché final, un goulot d'étranglement pour des séquences longues.

Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de génération ou de traduction ?

Sur le corpus d'actualités climatiques utilisé, les architectures récurrentes modélisent la séquence textuelle en factorisant la probabilité jointe par la règle de chaîne, chaque prédiction $P(x_t | h_{t-1})$ étant conditionnée par un état caché censé résumer tout l'historique. Les expériences montrent que le RNN simple, bien que théoriquement capable de cette modélisation, souffre en pratique d'une instabilité du gradient lors du BPTT (normes de gradient élevées et erratiques sans clipping), ce qui se traduit par une perplexité plus élevée et une convergence moins régulière que LSTM et GRU.

Le passage au LSTM et au GRU se justifie par leurs mécanismes de portes (forget, input, output pour LSTM ; reset, update pour GRU) qui régulent explicitement le flux d'information dans l'état caché/cellule, atténuant le vanishing gradient et permettant de mémoriser des dépendances à plus longue portée. Les résultats confirment des perplexités très proches et inférieures à celles du RNN simple, avec un GRU légèrement plus économe en paramètres.

Le passage à un schéma encodeur-décodeur (Seq2Seq) se justifie dès que la tâche change de nature : il ne s'agit plus de prédire un label ou le token suivant dans la même séquence, mais de **générer une séquence cible de longueur différente** à partir d'une séquence source (résumé, traduction). L'encodeur compresse la séquence source en un état caché, que le décodeur utilise comme point de départ pour générer token par token, avec le teacher forcing pour stabiliser l'entraînement. La qualité du décodage dépend ensuite de la stratégie utilisée : le beam search surpasse le décodage glouton (gain de score BLEU-1 mesuré expérimentalement) car il évite les choix gloutons prématurés en explorant plusieurs hypothèses simultanément.

En conclusion, la progression RNN → LSTM/GRU → Seq2Seq répond à des limites concrètes observées expérimentalement (instabilité du gradient, mémoire limitée, incapacité à générer des séquences de longueur arbitraire), chaque architecture étant une réponse architecturale ciblée à une limite de la précédente.

Synthèse transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Les trois parties de ce projet illustrent un même principe directeur : le deep learning ne propose pas une architecture universelle, mais une famille d'architectures dont le choix encode des **hypothèses structurelles** sur les données. Ces hypothèses, une fois intégrées dans l'architecture, réduisent l'espace de recherche du modèle et améliorent à la fois la performance et l'efficacité paramétrique.

Données tabulaires — MLP

Sur le dataset Air Quality, les variables d'entrée (polluants, météo, contexte) ne possèdent ni structure spatiale ni structure temporelle évidente : chaque variable est une mesure indépendante à un instant donné. Le MLP, en tant qu'approximateur universel de fonctions, est donc un choix raisonnable : il n'impose aucune hypothèse structurelle forte, mais peut apprendre des interactions non linéaires arbitraires entre variables — au prix d'un nombre de paramètres qui croît rapidement avec la dimension d'entrée et d'une absence d'exploitation explicite des corrélations entre variables.

Données images — CNN

Sur le dataset Intel Image Classification, les pixels possèdent une forte **dépendance locale** (un pixel est corrélé à ses voisins) et une **invariance par translation** (un même motif — un arbre, un bâtiment — peut apparaître n'importe où dans l'image). Le CNN encode directement ces deux hypothèses via la localité des filtres et le partage des poids, ce qui explique son avantage décisif en accuracy et en efficacité paramétrique par rapport au MLP, comme démontré expérimentalement dans la Partie II.

Données séquentielles — RNN/LSTM/GRU/Seq2Seq

Sur le dataset Climate Change News Articles, les tokens d'une phrase possèdent une **dépendance séquentielle ordonnée** : le sens d'un mot dépend de son contexte précédent (et parfois suivant). Les architectures récurrentes encodent cette hypothèse via un état caché mis à jour pas à pas, avec des mécanismes de portes (LSTM, GRU) pour gérer les dépendances à longue portée. Lorsque la tâche nécessite de produire une séquence cible de longueur différente de la séquence source (résumé), le schéma encodeur-décodeur (Seq2Seq) ajoute une hypothèse supplémentaire : la séquence source peut être compressée en une représentation fixe, à partir de laquelle la séquence cible est générée de manière autorégressive.

Tableau de synthèse

Dimension	MLP (Partie I)	CNN (Partie II)	RNN/LSTM/GRU/Seq2Seq (Partie III)
Hypothèse structurelle encodée	Aucune (approximateur universel)	Localité spatiale + partage des poids	Dépendance séquentielle ordonnée + mémoire
Unité de calcul de base	Couche dense (Linear)	Filtre de convolution	Cellule récurrente (RNN/LSTM/GRU)

Type de sortie	Vecteur de scores (classification)	Vecteur de scores (classification)	Séquence (génération) ou vecteur (classification)
Principal risque	Sur-paramétrisation, ignore les corrélations structurées	Sur-apprentissage si capacité trop élevée sans régularisation	Vanishing/exploding gradients (BPTT)

Tableau 9 — Synthèse comparative des trois architectures étudiées.

En définitive, l'apprentissage supervisé reste, dans les trois cas, un problème d'optimisation d'une fonction de perte par descente de gradient — mais la **forme de la fonction** (l'architecture) doit refléter la géométrie, la dépendance locale, la temporalité et la représentation propres aux données, sous peine de gaspiller la capacité du modèle à réapprendre des invariances qui auraient pu être directement encodées dans l'architecture.

Conclusion générale

Ce projet a permis de mettre en œuvre, sur trois jeux de données réels issus de Kaggle et unifiés par la thématique de l'environnement et du climat, les trois grandes familles d'architectures de deep learning étudiées dans le module : le perceptron multicouche (MLP), le réseau de neurones convolutionnel (CNN) et les architectures récurrentes (RNN, LSTM, GRU, Seq2Seq).

Sur le plan théorique, ce travail a permis d'approfondir la compréhension de concepts fondamentaux — graphe de calcul et rétropropagation, `state_dict` et persistance des modèles, stratégies d'initialisation, corrélation croisée 2D et calculs dimensionnels, modèle de langage probabiliste, BPTT et gradient clipping, teacher forcing et stratégies de décodage — en les reliant systématiquement à des observations expérimentales concrètes.

Sur le plan pratique, chaque partie a suivi une démarche rigoureuse : préparation des données sans fuite d'information, implémentation PyTorch commentée (avec, pour la Partie II, une validation numérique des implémentations manuelles face aux couches PyTorch), expérimentation comparative systématique (initialisations, architectures, stratégies de décodage), et évaluation à l'aide de métriques adaptées à chaque tâche.

La synthèse transversale a permis de relier ces trois expériences à un principe unificateur : le choix d'une architecture de deep learning revient à choisir quelles hypothèses structurelles sur les données seront encodées directement dans le modèle (absence de structure pour le MLP, localité et invariance par translation pour le CNN, dépendance séquentielle et mémoire pour les architectures récurrentes). Cette compréhension constitue, au-delà des résultats numériques obtenus, l'acquis principal de ce projet de fin de module.

Les pistes d'amélioration identifiées incluent : l'utilisation d'images en couleur à plus haute résolution avec augmentation de données pour la Partie II, l'intégration d'un mécanisme d'attention dans le système Seq2Seq de la Partie III, et l'exploration de modèles à base d'arbres (Random Forest, XGBoost) en complément du MLP pour la Partie I, afin de mieux exploiter les corrélations structurées entre variables tabulaires.

Annexes et références

Annexe A — Datasets utilisés (Kaggle)

Partie	Dataset Kaggle	Identifiant
I	Air Quality and Pollution Assessment	mujtabamatin/air-quality-and-pollution-assessment
II	Intel Image Classification	puneet6060/intel-image-classification
III	Climate Change News Articles	amritpal333/climate-news-articles

Note : pour chaque dataset, un mécanisme de génération synthétique calibrée ($SEED=42$) est intégré au notebook correspondant, garantissant son exécutabilité intégrale même en l'absence de credentials Kaggle ou d'accès réseau.

Annexe B — Liste des figures générées

- Partie I : `init_comparison.png`, `final_loss_curve.png`, `confusion_matrix_mlp.png`, `roc_curves_mlp.png`
- Partie II : `sample_images.png`, `cnn_loss_curve.png`, `ablation_study.png`, `feature_maps_block1.png`, `learned_filters_block1.png`, `roc_mlp_vs_cnn.png`, `confusion_matrix_cnn.png`
- Partie III : `rnn_lstm_gru_comparison.png`, `gradient_clipping.png`, `confusion_matrix_gru.png`, `seq2seq_loss.png`, `bleu_comparison.png`

Annexe C — Références bibliographiques

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*.
- Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.

Fin du rapport — EMSI Casablanca, Année universitaire 2025-2026 — Réalisé par Imane Bellghzal.